SECURITY IMPLEMENTATION

1. Overview

This document presents the identification and mitigation of three critical web application vulnerabilities in the Secure Patient Record System. The application was initially developed in a vulnerable state to demonstrate real-world security flaws and was later secured using appropriate countermeasures.

The vulnerabilities analyzed include:

- Insecure Direct Object Reference (IDOR)
- Cross-Site Request Forgery (CSRF)
- Clickjacking

Each vulnerability is demonstrated with its exploitation method followed by its secure implementation and results.

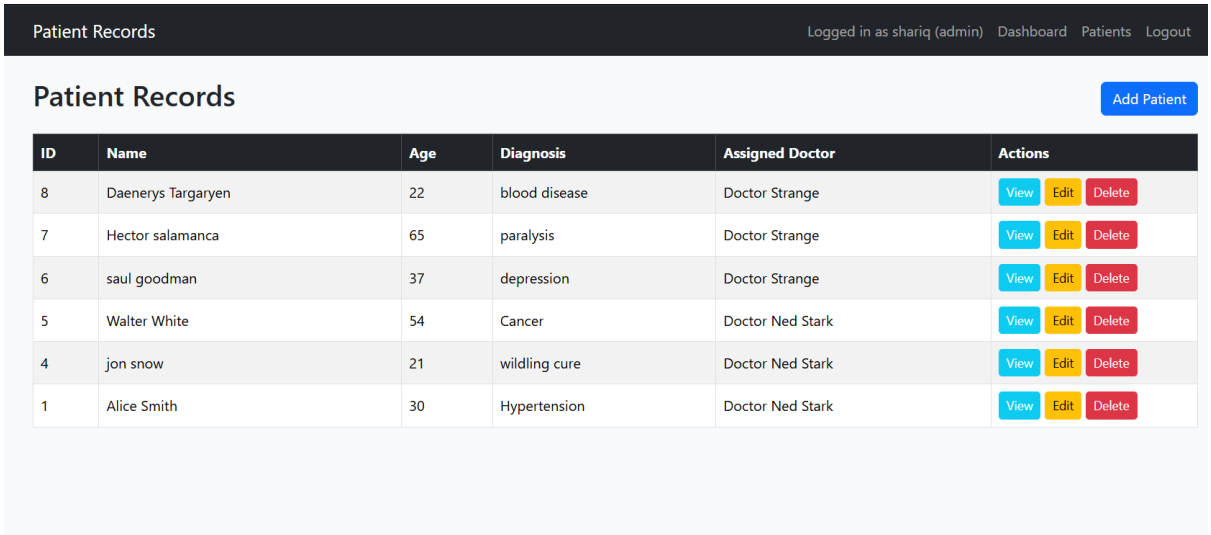
2. Insecure Direct Object Reference (IDOR)

2.1 Vulnerable Scenario

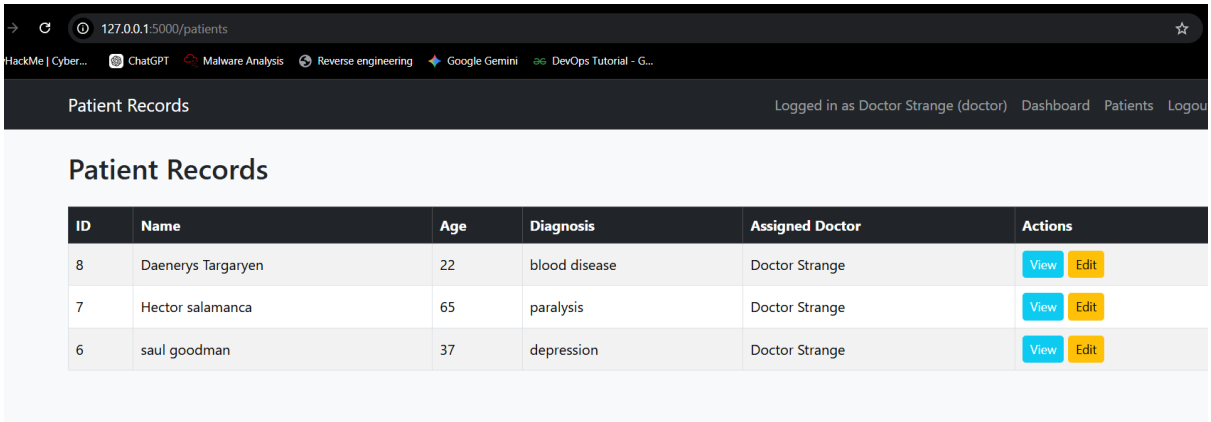
The application allowed access to patient records using a patient ID in the URL without verifying whether the logged-in doctor was authorized to access that specific record.

2.2 Evidence (Before Fix)

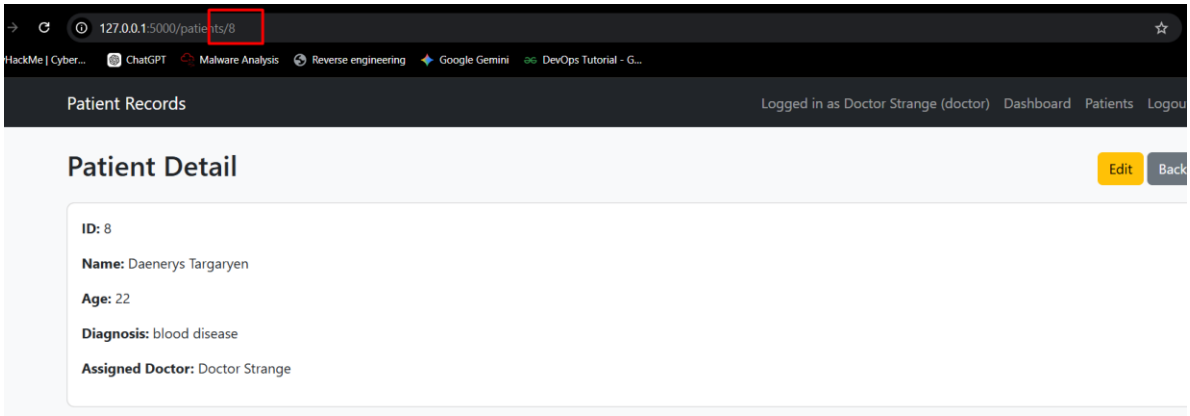
Doctor logs into the system and views assigned patients.



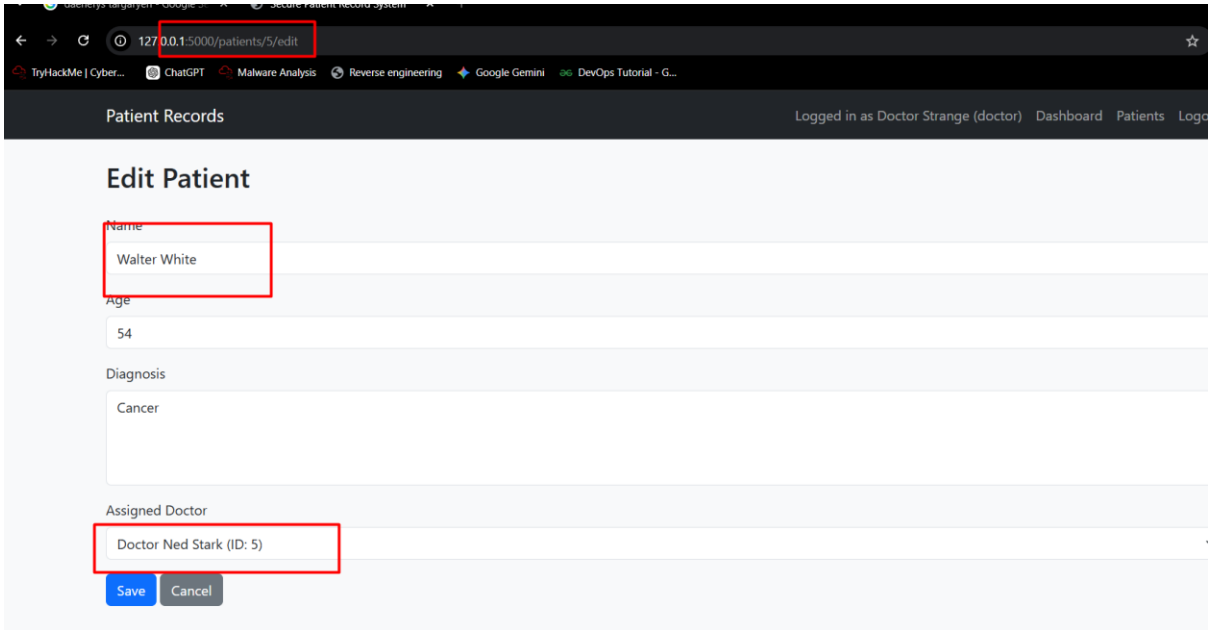
A patient record is accessed using its unique ID in the URL.



The ID in the URL is manually modified to another patient belonging to a different doctor.



The application successfully loads another doctor’s patient record.



2.3 Impact

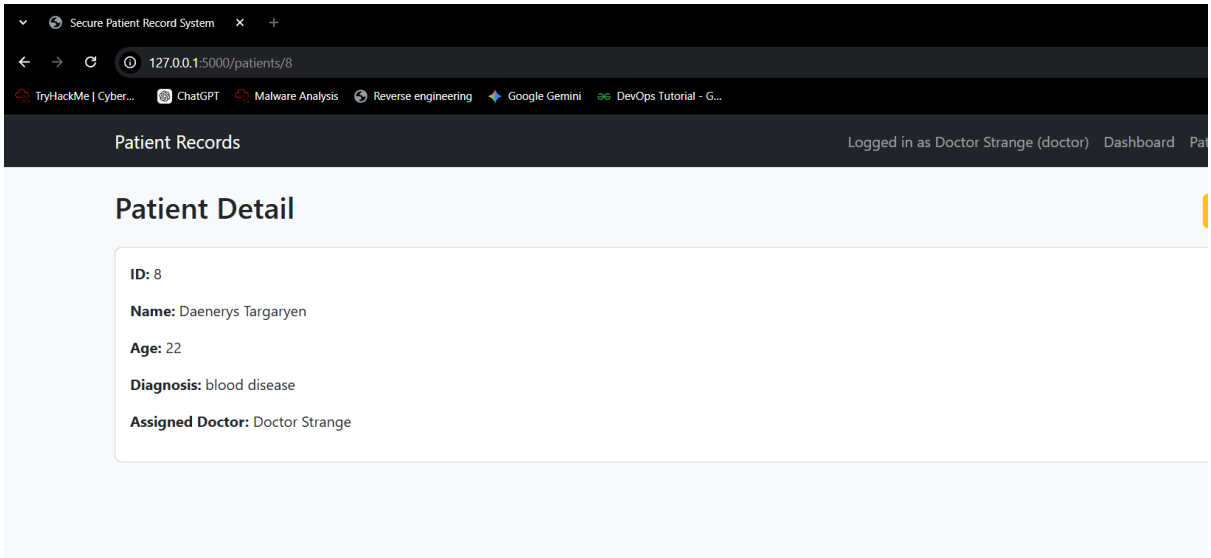
This vulnerability allows unauthorized access and modification of sensitive patient data, violating confidentiality and integrity.

2.4 Secure Implementation

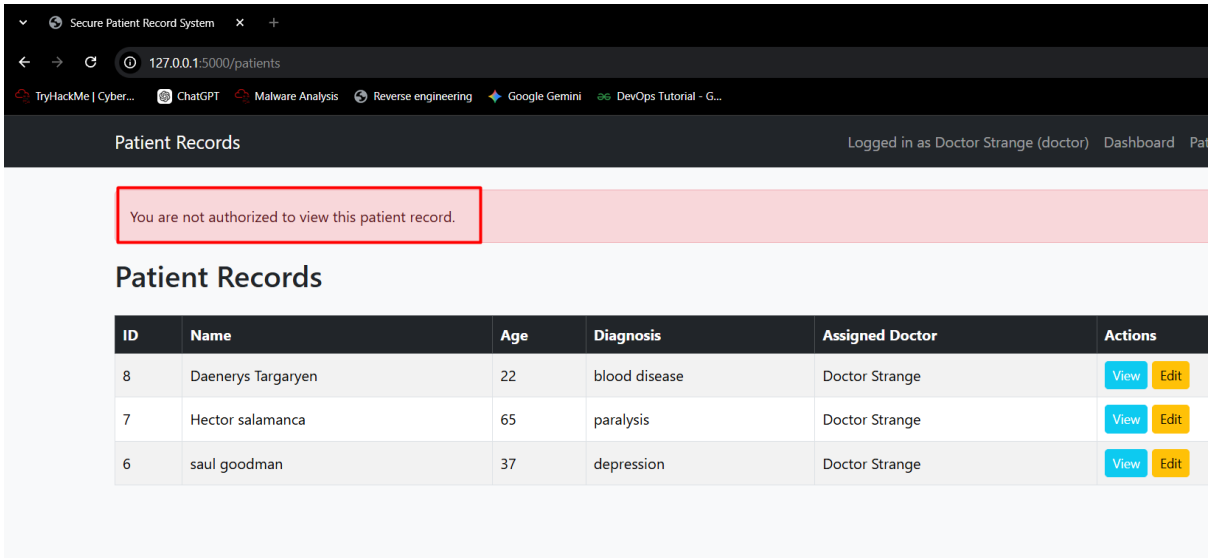
Object-level authorization checks were implemented to ensure that only the assigned doctor or an administrator can access a patient record.

2.5 Evidence (After Fix)

Attempting to access another doctor’s patient record.



The application blocks access and displays an error message.



2.6 Result

Unauthorized access is now restricted, and only authorized users can access patient records.

3. Cross-Site Request Forgery (CSRF)

3.1 Vulnerable Scenario

The application did not include CSRF protection, allowing attackers to send unauthorized requests on behalf of authenticated users.

3.2 Evidence (Before Fix)

The application form does not contain a CSRF token.

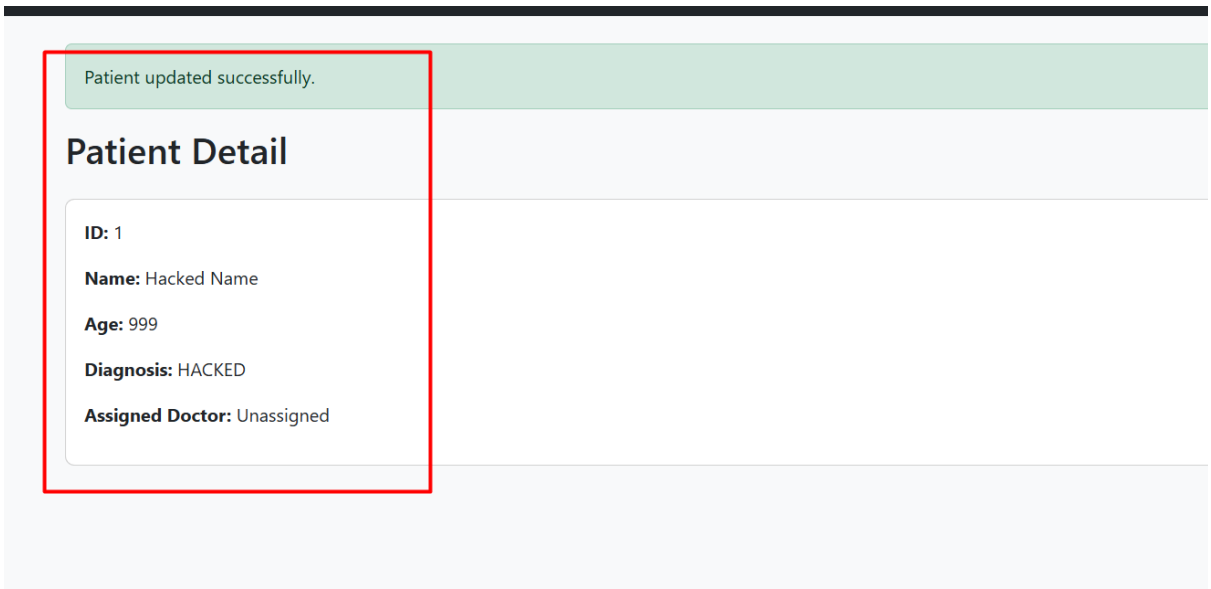
```
<main class="container py-4">
  <h2 class="mb-4">Edit Patient</h2>
  <form method="POST" action="/patients/5/edit"> == $0
    <div class="mb-3">…</div>
    <div class="mb-3">…</div>
    <div class="mb-3">
      <label class="form-label">Diagnosis</label>
      <textarea name="diagnosis" class="form-control" rows="4" required>Cancer
      </textarea>
    </div>
    <div class="mb-3">…</div>
    <button type="submit" class="btn btn-primary">Save</button>
    <a href="/patients" class="btn btn-secondary">Cancel</a>
  </form>
```

A malicious CSRF attack script is created.

```
File    Edit    View

<html>
<body>

<h2>CSRF Attack</h2>

<form action="http://127.0.0.1:5000/patients/1/edit" method="POST">
  <input type="hidden" name="name" value="Hacked Name">
  <input type="hidden" name="age" value="999">
  <input type="hidden" name="diagnosis" value="HACKED">
</form>

<script>
  document.forms[0].submit();
</script>

</body>
</html>
```

The script executes automatically and submits a request.

The patient data is modified without user interaction.



3.3 Impact

Attackers can perform unauthorized actions such as modifying or deleting patient data using the victim's session.

3.4 Secure Implementation

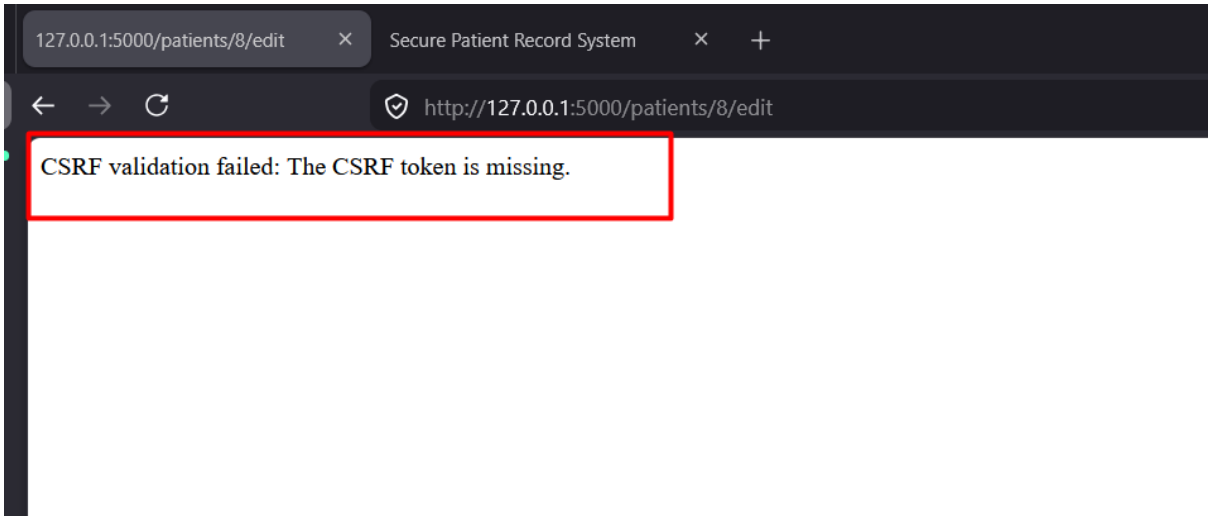
CSRF protection was implemented by adding CSRF tokens to all forms and validating them on the server. Requests without valid tokens are rejected.

3.5 Evidence (After Fix)

Forms now include CSRF tokens.

```
<html lang="en"> event scroll
  <head> ... </head>
  <body> overflow
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark"> flex
      <div class="container"> ... </div> flex
    </nav>
    <main class="container py-4"> overflow
      <h2 class="mb-4">Edit Patient</h2>
      <form method="POST" action="/patients/8/edit">
        <input type="hidden" name="csrf_token"
        value="ImU3MzRkMjM1MTE4YWM3NDFmYTQzNjRlYTIyMDlkZTA0OWYzM2E4ZWMi.adeW0Q.EM_St-01cAxjR2_RgeFMfau6RVA">
        <div class="mb-3"> ... </div>
        <div class="mb-3"> ... </div>
        <div class="mb-3"> ... </div>
        <div class="mb-3"> ... </div>
        <button class="btn btn-primary" type="submit">Save</button>
        whitespace
        <a class="btn btn-secondary" href="/patients">Cancel</a>
      </form>
    </main>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  </body>
</html>
```

Removing the CSRF token results in request failure.



3.6 Result

All forged or unauthorized requests are blocked, ensuring secure request handling.

4. Clickjacking

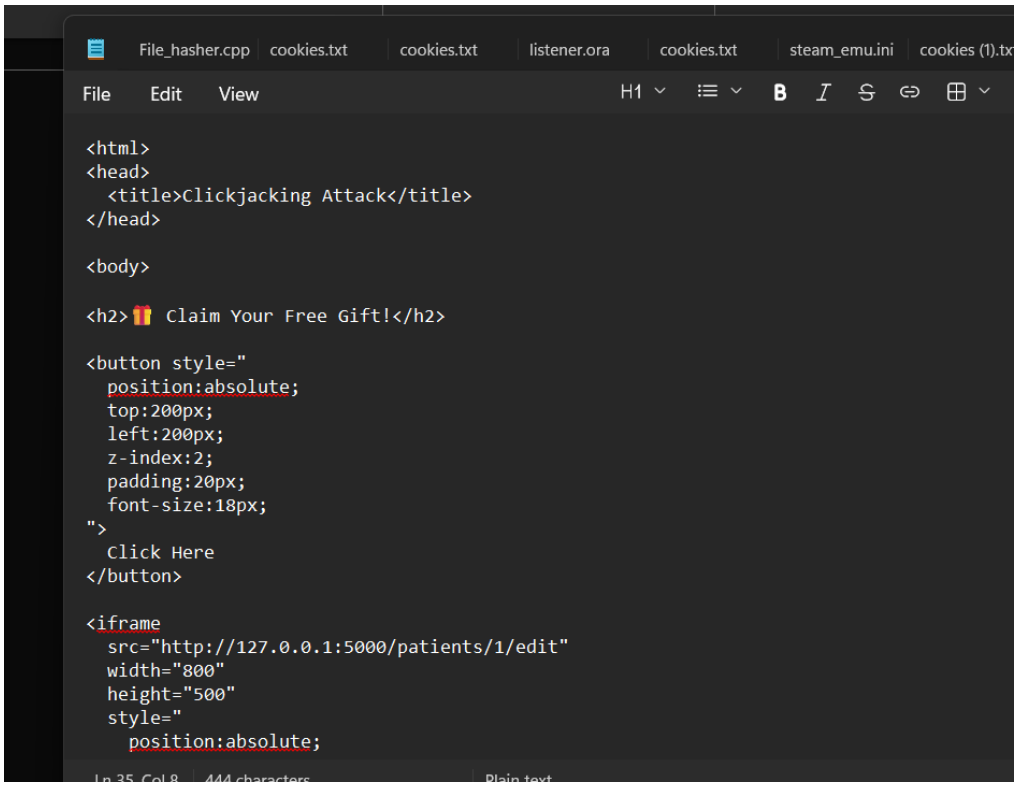
4.1 Vulnerable Scenario

The application did not include anti-framing security headers, allowing it to be embedded in an iframe on an external malicious page.

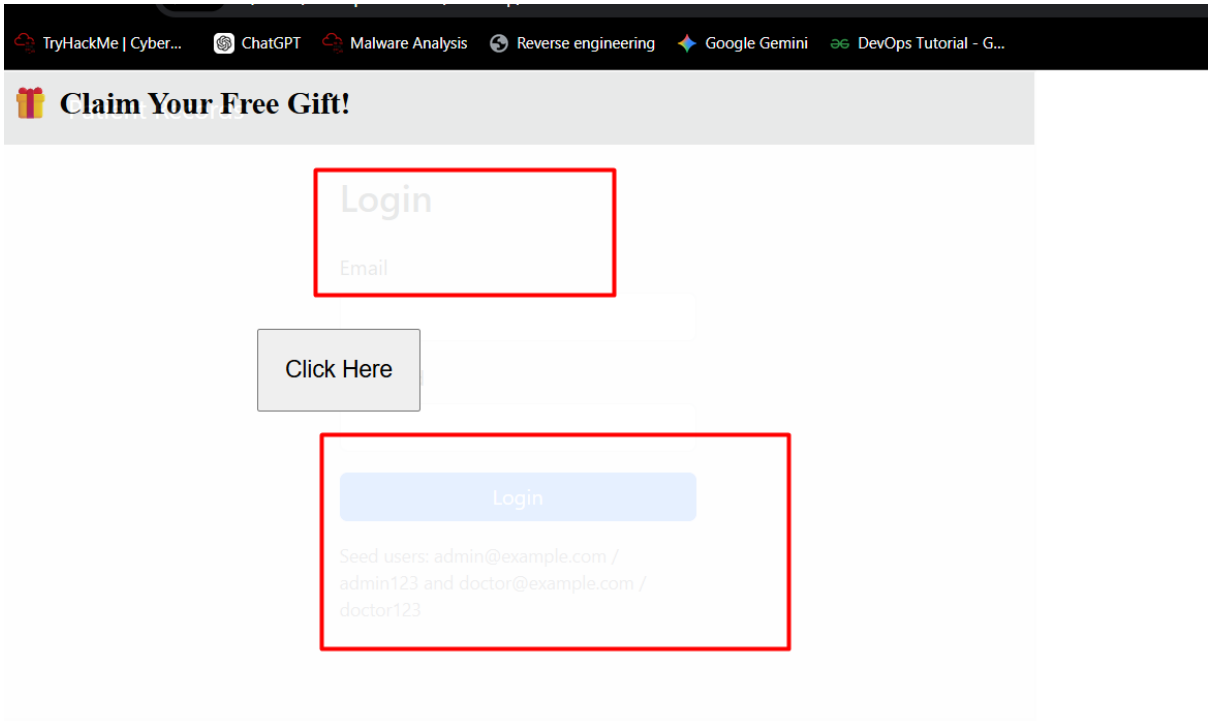
4.2 Evidence (Before Fix)

Response headers do not contain security protections.

The application is successfully embedded inside an iframe.



A fake UI overlay is placed over the application.



4.3 Impact

Users can be tricked into performing unintended actions through deceptive UI overlays.

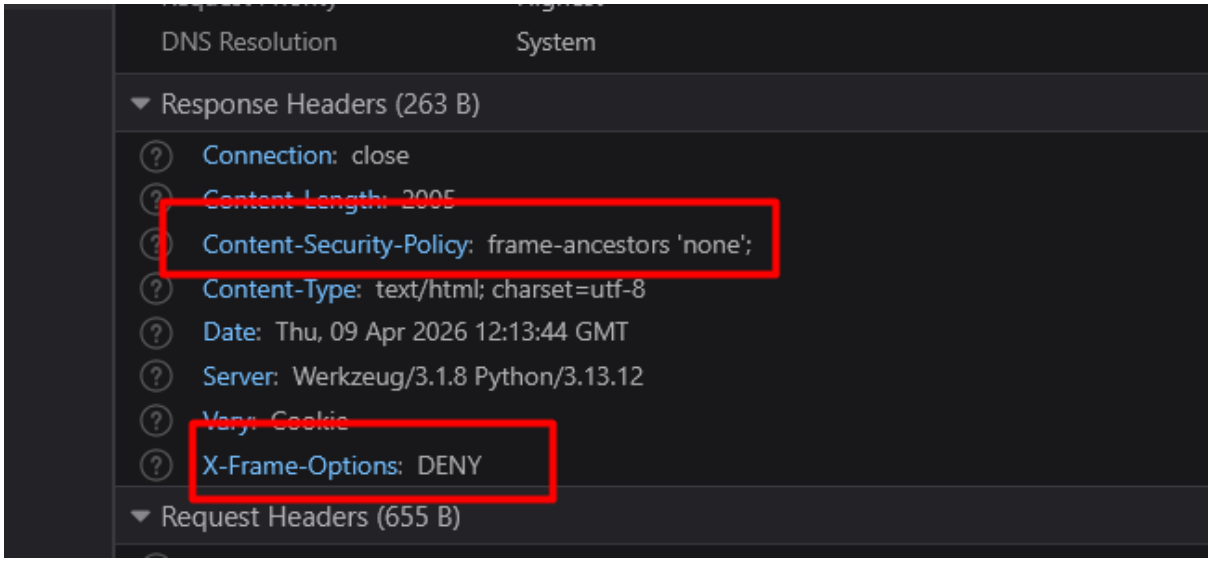
4.4 Secure Implementation

The following security headers were added:

- X-Frame-Options: DENY
- Content-Security-Policy: frame-ancestors 'none'

4.5 Evidence (After Fix)

Security headers are now present in the response.



Attempting to embed the application in an iframe is blocked.

4.6 Result

The application can no longer be embedded in external pages, effectively preventing clickjacking attacks.

5. Summary

All identified vulnerabilities were successfully mitigated using secure coding practices. The implementation of authorization checks, CSRF protection, and security headers significantly improved the overall security of the application.

The system now ensures:

- Proper access control for patient data
- Protection against unauthorized request forgery
- Prevention of clickjacking attacks